Hands-On 1: Tree Algorithms

Competitive Programming and Contests (2025/2026) – Lorenzo Ferrari

1 Overview

The assignment involves implementing algorithms on a binary tree represented via index-based adjacency in a flat vector. This report outlines the core algorithmic logic for `is_binary` and `max_path`. This report has been generated by Antigravity with Gemini 3.8 Flash and modified by the author.

2 `is_binary`: BST Property Validation

The objective is to verify whether the binary tree satisfies the Binary Search Tree ordering property: every node's key must be greater than or equal to all keys in its left subtree and less than or equal to all keys in its right subtree.

- **Algorithmic Logic**:
  - The algorithm performs a **recursive traversal** maintaining an admissible value interval $[L, R]$ for each node.
- **Recursion Invariant**:
  - The root is initially bounded by `[u32::MIN, u32::MAX]`.
  - An empty subtree trivially satisfies the property.
  - For any node with key k, the algorithm first checks $L \leq k \leq R$. If violated, the tree is not a BST.
  - It then recurses on the left child with the tightened upper bound $[L, k]$ and on the right child with the tightened lower bound $[k, R]$. Both subtrees must be valid.
- **Complexity**:
  - $\mathcal{O}(N)$ time since every node is examined once; $\mathcal{O}(H)$ auxiliary stack space, where H is the tree height.

3 `max_path`: Maximum Leaf-to-Leaf Path Sum

The goal is to compute the maximum sum along any simple path connecting two leaves in the tree.

- **Algorithmic Logic**: Any simple path connecting two leaves has a unique highest node, their Lowest Common Ancestor, where the path turns. The algorithm uses a recursive approach to compute, at each node v, the maximum sum along any simple path connecting two leaves whose LCA is v. There is no need for caching or memoizing any state: values are computed directly on the call stack and propagated upward.
- **Recursive Return Values**: Each recursive call returns a pair of values for the current subtree:
  1. **`max_leaf_to_leaf`**: the maximum path sum connecting two leaves within the subtree.
  2. **`max_node_to_leaf`**: the maximum path sum from the current node down to a leaf in its subtree.
- **Combination Step**:
  - **Leaf Node**: Both returned values are simply the node's key k.
  - **Full Node (two children)**: The node can bridge a leaf in the left subtree to a leaf in the right subtree with total sum:

    $$k + \text{left}_{\text{to_leaf}} + \text{right}_{\text{to_leaf}}$$

    The best leaf-to-leaf path is the maximum of this bridge and the best leaf-to-leaf paths returned by both children. The node-to-leaf path is $k + \max(\text{left}_{\text{to_leaf}}, \text{right}_{\text{to_leaf}})$.
  - **Unary Node (single child)**: An LCA turn between two leaves cannot occur at this node. The call propagates the child's best leaf-to-leaf path, while extending the node-to-leaf path to $k + \text{child}_{\text{to_leaf}}$.
- **Complexity**: $\mathcal{O}(N)$ time and $\mathcal{O}(H)$ auxiliary stack space, visiting each node exactly once.